ULPGC
Universidad de
Las Palmas de
Gran Canaria

Big Data

Degree in Data Science and Engineering

School of Computer Engineering

Academic year 2026–2027

Individual Assignment 1

Basic Matrix Multiplication in Python, Java and C

How matrix size affects the runtime and memory use of a basic triple-loop matrix multiplication

---

Name

Claudia Álvarez González

Repository:

https://github.com/Claudia1771/bigdata-matrix-multiplication

(folder `assignment1`)

October 14, 2026

Contents

Abstract **1**

1 Introduction and Objective **2**
1.1 Research question 2
1.2 Hypotheses 2

2 Background **3**
2.1 The basic algorithm 3
2.2 Memory access pattern 3
2.3 Execution models 3
2.4 Floating-point arithmetic 4

3 Methodology **5**
3.1 Implementations 5
3.2 Inputs 5
3.3 Correctness checks 5
3.4 Measurement protocol 6
3.5 Environment 7

4 Results **8**
4.1 Correctness 8
4.2 Hybrid processor and CPU affinity 8
4.3 Execution time 9
4.4 Variability 11
4.5 Memory 12
4.6 Limits 13

5 Discussion **14**
5.1 Does the time grow as n^3? (H1) 14
5.2 The power-of-two effect 14
5.3 Python versus compiled languages (H2) 15
5.4 Java versus C (H3) 15
5.5 Memory (H4) 15
5.6 Variability and the hybrid processor 16
5.7 Limitations 16

6 Conclusions **17**

AI Usage Statement **18**

Reproducibility Materials **19**

References **20**

List of Figures

4.1 Preliminary run without CPU affinity versus final run pinned to a performance core. (a) Median time for $n = 1024$, with the range from the fastest to the slowest repetition. (b) Ratio between the slowest and the fastest repetition of every configuration ($n = 64$–2048). 9
4.2 Median execution time of one multiplication versus n (log–log scale; bars show the IQR). The dashed line has slope 3 (cubic growth). The point just to the left of 1024 is $n = 1000$. The cross marks the Python timeout. 10
4.3 Median time divided by n^3 (nanoseconds per multiply-add). A horizontal line would mean exactly cubic growth. 11
4.4 Each measured repetition divided by the median of its configuration. 12
4.5 Peak working set of the process versus n, with the storage estimate of three `float64` matrices. 13

List of Tables

3.1 Implementations, storage layouts and fixed execution settings. 5
3.2 Hardware and software environment of the reported run. 7

4.1 Speed of the logical processors (median of 3 runs of the same Python loop pinned to each processor)..... 8
4.2 Execution time of one multiplication: median and interquartile range [Q_1, Q_3] in seconds, and number of measured repetitions r. All processes pinned to a performance core..... 10
4.3 Observed time ratio between consecutive sizes (cubic model in brackets), time per multiply-add in nanoseconds, and time relative to C for the same n. 11
4.4 Peak working set of the process (MiB), including the language runtime. The Python value for $n = 2048$ corresponds to the run stopped by the time limit..... 12

List of Algorithms

1 Basic i-j-k matrix multiplication 3

Abstract

This study measures how the matrix size n affects the execution time and memory use of the same basic i-j-k triple-loop matrix multiplication implemented in Python, Java and C. The three programs read identical `float64` input matrices, produce bit-identical results, and were run with a documented protocol: shared inputs, correctness tests, warm-up runs, five repetitions (three for runs of more than 20 s), median and interquartile range, a 300 s time limit, and every process pinned to one performance core of a hybrid laptop processor. For $n = 1024$ the median time of one multiplication was 72.8 s in Python, 1.19 s in Java and 2.37 s in C; Python could not complete $n = 2048$ within the time limit, while Java and C needed 69.7 s and 32.2 s. The growth follows the cubic model only while the data fits in the caches: for larger sizes the cost per multiply-add increases, and the fitted exponents over $n = 256$–2048 are between 3.4 and 4.1. The contiguous C layout suffers a strong power-of-two effect (going from $n = 1000$ to 1024 multiplies its time by 3.6, against an expected 1.07), which the Java and Python layouts do not show. A preliminary run without CPU affinity was up to 2.7 times slower and much more variable, showing that the core assignment of a hybrid processor must be controlled. These conclusions are limited to the tested implementations, settings and machine.

1. Introduction and Objective

Dense matrix multiplication is a basic building block of scientific computing, computer graphics and machine learning. For two $n \times n$ matrices the straightforward algorithm performs n^3 multiply-add operations, so its cost grows cubically with the matrix side n. Asymptotic complexity, however, says nothing about the constant factors that determine the actual running time on a given machine. These depend on how the language executes the code (interpretation, just-in-time or ahead-of-time compilation), how the matrices are stored in memory and how the algorithm interacts with the memory hierarchy [2, 3].

1.1. Research question

How does the matrix size n affect the execution time and the memory use of the same basic triple-loop matrix multiplication implemented in Python, Java and C, under fixed and documented execution conditions?

The factors changed are the matrix size n (the side of an $n \times n$ square matrix) and the implementation language. The outcomes measured are the execution time of one multiplication and the peak memory of the process.

1.2. Hypotheses

The following predictions were stated before running the experiment:

H1. For each language the execution time grows approximately as n^3, so doubling n multiplies the time by about 8.

H2. Python is one to two orders of magnitude slower than Java and C, because the interpreter executes every operation on boxed objects.

H3. Java and C are within a small factor of each other once the JIT compiler has optimised the loop.

H4. Memory grows as n^2, with a fixed runtime overhead that is largest for Java and a per-element overhead that is largest for Python.

2. Background

2.1. The basic algorithm

Given A and B, each element of $C = AB$ is $C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}$ [1]. Algorithm 1 shows the classic i-j-k triple loop used in this study. It performs exactly n^3 multiplications and n^3 additions.

Algorithm 1 Basic i-j-k matrix multiplication

Require: $A, B \in \mathbb{R}^{n \times n}$
Ensure: $C = AB$
allocate $C \in \mathbb{R}^{n \times n}$
for $i \leftarrow 1$ **to** n **do**
    for $j \leftarrow 1$ **to** n **do**
        $s \leftarrow 0$
        for $k \leftarrow 1$ **to** n **do**
            $s \leftarrow s + A_{ik} \cdot B_{kj}$
        end for
        $C_{ij} \leftarrow s$
    end for
end for
return C

2.2. Memory access pattern

In the i-j-k order the innermost loop reads one row of A sequentially and one column of B with a stride of n elements. With row-major storage, each access to B touches a different cache line, so for large n most accesses to B miss in the cache [3, 4, 5]. When the row length is a power of two, the elements of a column map to a small number of cache sets and can evict each other (conflict misses) [3, 5]. Optimised libraries avoid these effects with blocking and vectorisation [6]; they are outside the scope of this controlled comparison and will be studied in Assignment 2.

2.3. Execution models

CPython interprets bytecode; every `float` is a heap object, and every arithmetic operation involves dynamic type dispatch and reference counting. Java code is first interpreted and then compiled at run time by the HotSpot JIT compiler, which is why warm-up runs are recommended before measuring [7]. C is compiled ahead of time to native code.

Leiserson et al. [2] used this same triple loop to show differences of several orders of magnitude between Python and C.

2.4. Floating-point arithmetic

Floating-point addition is not associative, so a different summation order can change the last bits of a result. For an inner product of length n, the error with respect to the exact value is bounded by about $nu \sum_k |A_{ik} B_{kj}|$, where $u \approx 1.1 \times 10^{-16}$ is the unit roundoff of `float64` [9]. Since all three implementations perform the same operations in the same order, and Java uses strict IEEE 754 semantics by default since version 17 [10], they are expected to produce bit-identical results.

3. Methodology

3.1. Implementations

The three programs implement Algorithm 1 with a scalar accumulator, use 64-bit floating-point numbers and do not call any matrix library. They differ only in the unavoidable, idiomatic storage layout of each language (Table 3.1). All implementations are single-threaded. The C program creates no additional threads; the JVM creates internal threads for JIT compilation and garbage collection, which in this study share the single core assigned to the process.

Table 3.1: Implementations, storage layouts and fixed execution settings.

Language	**Storage of each matrix**	**Execution settings (fixed for the whole study)**
Python	List of n lists of Python `float` objects (each element is a pointer to a 24-byte object)	CPython 3.13.0, default interpreter (experimental JIT not enabled)
Java	`double[][]`: an array of n independent row arrays of primitive doubles	JDK 21.0.5 HotSpot, default tiered JIT, serial collector, `-Xmx4g`
C	One contiguous row-major `double` array of n^2 elements	GCC 16.2.0 (MinGW-w64), `-O2 -std=c11`, no `-march` flag

3.2. Inputs

To guarantee that all languages multiply the same logical matrices, the inputs are generated once by `generate_inputs.py` and read by the three programs. For each size, A and B are drawn with NumPy's `default_rng` (PCG64) using seed $42 + n$, uniform values in $[0, 1)$, type `float64`, and stored as raw little-endian row-major binary files. A manifest records the seed and the SHA-256 hash of every file.

The declared size sequence is $n \in \{64, 128, 256, 512, 1000, 1024, 2048\}$: powers of two, so that consecutive sizes should differ by a factor of 8 in time, plus $n = 1000$ next to 1024 to check whether the power-of-two size has an effect of its own.

3.3. Correctness checks

Before benchmarking, `verify.py` runs every implementation on:

- 1×1 and 2×2 matrices with known integer products (exact comparison);
- $AI = A$ and $A\,0 = 0$ for a random 8×8 matrix A (exact comparison);
- random matrices with $n \in \{1, 2, 3, 17, 64, 100\}$, compared with `numpy.matmul` as the trusted reference using $\text{rtol} = \text{atol} = 10^{-12}$.

This tolerance is about 100 times larger than the error bound nu for $n = 100$ and values in $[0, 1)$ (no cancellation is possible because all products are non-negative), but small enough to detect any indexing error. The outputs of the three languages are also compared with each other bit by bit.

In addition, during the benchmark every multiplication returns a checksum: the sequential sum of all elements of C, reported as the raw bits of the double. This guarantees that the result is consumed, so the compiler cannot remove the computation, and allows checking that every repetition in every language produced the same result.

3.4. Measurement protocol

- **Timer boundary (kernel measurement).** The timer covers only the multiplication function, including the allocation of the output matrix C (an $O(n^2)$ cost, negligible compared with the $O(n^3)$ loop). Reading the input files, process start-up and the checksum are outside the timed region. Monotonic timers are used: `time.perf_counter` (Python), `System.nanoTime` (Java) and `clock_gettime(CLOCK_MONOTONIC)` (C).
- **Processes, warm-up and repetitions.** Each (language, n) configuration runs in a fresh process that loads the inputs, executes the warm-up multiplications (not reported) and then the measured repetitions. Default: 5 measured repetitions, with 3 warm-ups for Java (JIT compilation) and 1 for Python and C (caches and memory allocation). Costly configurations, defined as those whose predicted time (median of the previous size multiplied by $(n/n_{\text{prev}})^3$) exceeds 20 s, use 3 repetitions and 0 warm-ups (1 for Java), to keep the study feasible. For runs of minutes, the JIT compiles the loop during the first fraction of a second, so its effect is negligible.
- **Very short configurations.** The shortest multiplication (C, $n = 64$) takes about 78 μs, which is several hundred times longer than the sub-microsecond resolution of the monotonic timers used. Each multiplication is therefore timed individually instead of in batches; the IQR of the $n = 64$ configurations, below 3% of the median in the three languages, confirms that the timer resolution does not limit these measurements.
- **CPU affinity and priority.** The processor is hybrid, and a probe that runs the same CPU-bound loop pinned to each logical processor (`core_probe.py`, Section 4.2) shows that the efficient cores are about 2–2.6 times slower than the performance cores. A preliminary run without affinity showed repetitions up to 6.6 times

slower than others of the same configuration. Therefore, every benchmark process is created already pinned to logical processor 12 (a performance core) with `HIGH` priority class. Because the JVM detects a single available processor, the garbage collector is fixed explicitly to the serial collector (`-XX:+UseSerialGC`), and JIT compilation shares the same core as the measured thread.

- **Budgets.** Maximum time of one multiplication: 300 s. Memory budget: 4 GiB per process. A configuration that exceeds a limit is stopped and recorded as a timeout or memory failure, and the larger sizes of that language are not attempted. No time is invented for failed configurations.
- **Summary statistics.** All raw measurements are kept in `raw_measurements.csv`, where every row is labelled with the language, the matrix size n, the numeric type (`float64`), the kind of run (warm-up or measured) and the repetition index. Every run used one thread pinned to logical processor 12; this is recorded in `environment.json` and in the `threads` and `cpu_affinity` columns of `summary.csv`. Tables report the median, the first and third quartiles (Q_1, Q_3) and the interquartile range (IQR), computed with linear interpolation.
- **Memory metric.** Peak working set (peak resident memory) of the whole process, read with the Win32 `GetProcessMemoryInfo` API after the process has finished. It includes the inputs, the result, temporary objects and the language runtime (interpreter or JVM), so it is not a measure of the matrix storage alone. To avoid measuring a launcher process, Java is started with the `java.exe` of the JDK. As a reference, the storage of three $n \times n$ `float64` matrices is $24n^2$ bytes.

3.5. Environment

Table 3.2: Hardware and software environment of the reported run.

Item	Value
CPU	Intel Core Ultra 7 155H (hybrid: 6 P-cores with 2 threads each, 8 E-cores, 2 LP E-cores)
Cores / logical processors	16 / 22
Cache	24 MB L3
RAM	16 GB
Operating system	Windows 11 (build 26200), laptop connected to the power supply
Python	CPython 3.13.0 (NumPy 2.2.6 only for inputs and reference)
Java	Oracle JDK 21.0.5 HotSpot, `-Xmx4g -XX:+UseSerialGC`
C compiler	GCC 16.2.0 (MinGW-w64 UCRT), `-O2 -std=c11`
CPU affinity	Logical processor 12 (performance core), `HIGH` priority class

The whole configuration is stored in `config.json` and the environment captured at run time in `environment.json`.

4. Results

All the numbers in this chapter come from `results/raw_measurements.csv`, `memory.csv` and `summary.csv`, produced by `analyze.py`. Following common recommendations for reporting performance measurements [8], the median is used as the central value and the interquartile range (IQR) as the measure of variability.

4.1. Correctness

All 30 tests (10 cases $\times$ 3 languages) passed. The exact cases (known 1×1 and 2×2 products, identity and zero matrix) gave exactly the expected values. For the random cases, the maximum absolute difference with respect to `numpy.matmul` grew with n, as expected from the error bound, up to 2.1×10^{-14} for $n = 100$, well inside the tolerance of 10^{-12}. The outputs of Python, Java and C were bit-identical in every test, and during the benchmark every repetition of every language produced the same checksum for each size. Therefore the three programs perform exactly the same floating-point operations in the same order, and the comparison is not affected by different arithmetic. The GCC vectorisation report (`-fopt-info-vec`) confirms that the C loop was not vectorised; the generated code uses scalar `mulsd` and `addsd` instructions.

4.2. Hybrid processor and CPU affinity

Table 4.1 summarises `core_probe.csv`: the same CPU-bound loop takes about 0.22–0.24 s on the 12 logical processors of the performance cores and 0.45–0.58 s on the efficient and low-power cores.

Table 4.1: Speed of the logical processors (median of 3 runs of the same Python loop pinned to each processor).

Logical processors	Core type	Median time (s)
0, 1, 10–19	Performance cores (6 cores, 2 threads each)	0.220–0.236
2–9	Efficient cores	0.447–0.534
20, 21	Low-power efficient cores	0.571–0.577

A first complete run without affinity (kept in `results/preliminary_run_unpinned`) produced repetitions of the same configuration that differed by factors of up to 6.6 (Python, $n = 512$) and 3.8 (C, $n = 2048$), and its medians were clearly higher: for $n = 1024$, 150.1 s (Python), 2.85 s (Java) and 6.32 s (C), against 72.8 s, 1.19 s and 2.37 s when the process

was pinned to a performance core (Figure 4.1). These preliminary values are not used in the rest of the report.

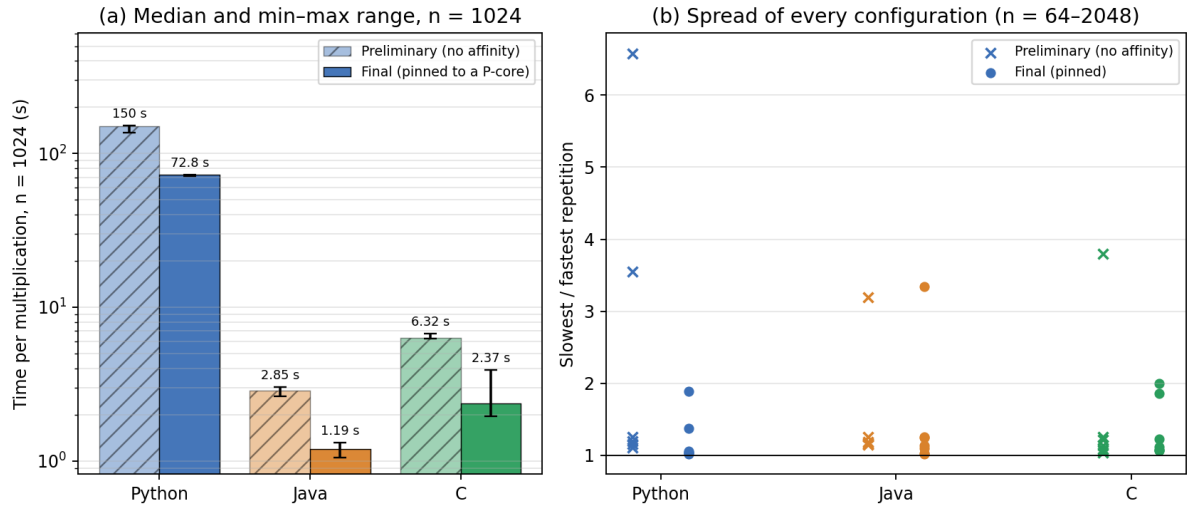


Figure 4.1: Preliminary run without CPU affinity versus final run pinned to a performance core. (a) Median time for $n = 1024$, with the range from the fastest to the slowest repetition. (b) Ratio between the slowest and the fastest repetition of every configuration ($n = 64$–2048).

Pinning reduced the medians for $n = 1024$ by a factor of 2.1 (Python), 2.4 (Java) and 2.7 (C). It also removed the most extreme spreads: in the preliminary run the slowest repetition was up to 6.6 times slower than the fastest, while in the final run this ratio is at most 2.0, except for the single Java outlier at $n = 128$ discussed in Section 4.4.

After the preliminary run, and before the final run, three changes were made to the protocol, all of which are reflected in Chapter 3: (1) every process was pinned to a performance core with high priority; (2) because the JVM then detects a single processor, its garbage collector was fixed explicitly to the serial collector instead of the default G1 collector; and (3) the threshold that defines a costly configuration was lowered from 60 s to 20 s. The last change was needed because Java with $n = 2048$ took between 153 and 242 s per multiplication in the preliminary run, far more than the 23 s predicted by the cubic model from $n = 1024$, so its eight runs (three warm-ups and five repetitions) were not feasible within the time budget. The final run was then executed once, completely, with the protocol of Chapter 3; no configuration was repeated or selected afterwards.

4.3. Execution time

Table 4.2 reports the median time of one multiplication and its quartiles, and Figure 4.2 plots them.

Table 4.2: Execution time of one multiplication: median and interquartile range $[Q_1, Q_3]$ in seconds, and number of measured repetitions r. All processes pinned to a performance core.

	Python		**Java**		**C**	
n	median $[Q_1, Q_3]$	r	median $[Q_1, Q_3]$	r	median $[Q_1, Q_3]$	r
64	0.00906 [0.00904, 0.00907]	5	0.000113 [0.000112, 0.000114]	5	0.0000777 [0.0000774, 0.0000786]	5
128	0.0689 [0.0689, 0.0694]	5	0.00105 [0.00099, 0.00108]	5	0.00179 [0.00176, 0.00188]	5
256	0.604 [0.603, 0.604]	5	0.00984 [0.00972, 0.00987]	5	0.0137 [0.0137, 0.0159]	5
512	10.60 [9.73, 11.31]	5	0.111 [0.111, 0.112]	5	0.152 [0.151, 0.152]	5
1000	71.31 [69.45, 82.17]	3	1.129 [1.020, 1.243]	5	0.659 [0.638, 0.889]	5
1024	72.80 [72.22, 72.89]	3	1.194 [1.147, 1.229]	5	2.373 [2.215, 2.944]	5
2048	timeout ($>$ 300 s)	0	69.73 [63.73, 69.99]	5	32.18 [31.48, 32.34]	5

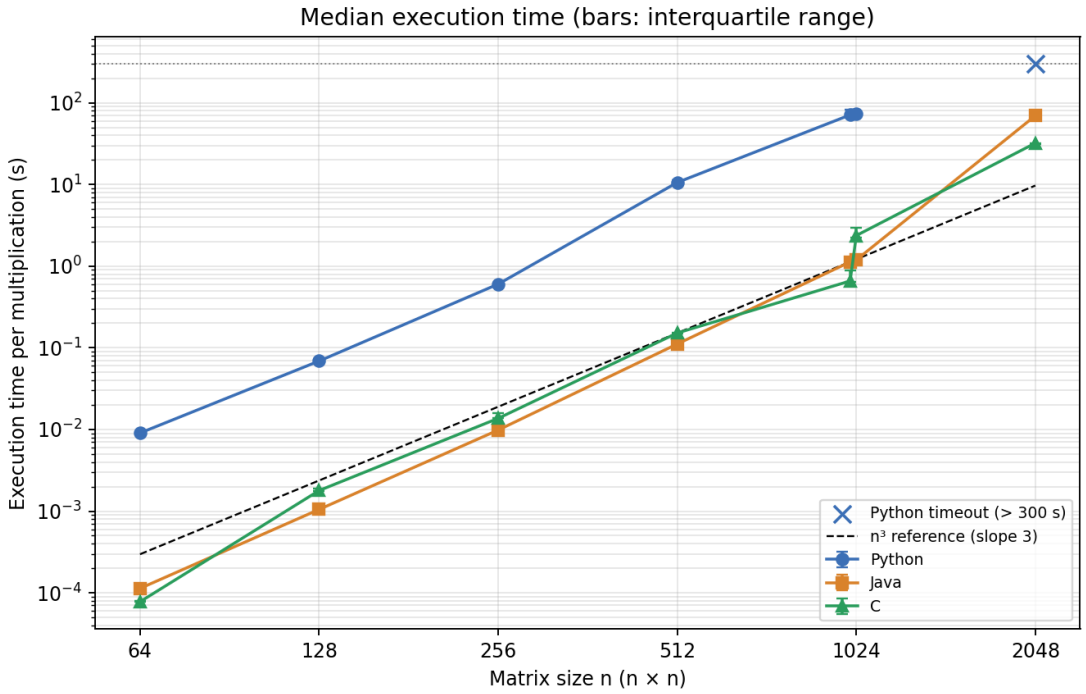


Figure 4.2: Median execution time of one multiplication versus n (log–log scale; bars show the IQR). The dashed line has slope 3 (cubic growth). The point just to the left of 1024 is $n = 1000$. The cross marks the Python timeout.

Table 4.3 compares the observed growth between consecutive sizes with the growth predicted by the cubic model, and shows the cost per inner-loop iteration (median time divided by n^3), which would be constant if the time grew exactly as n^3. Figure 4.3 plots this normalised cost.

Table 4.3: Observed time ratio between consecutive sizes (cubic model in brackets), time per multiply-add in nanoseconds, and time relative to C for the same n.

		Ratio to previous size			**ns per multiply-add**			**Time / C time**	
n	(cubic)	Python	Java	C	Python	Java	C	Python	Java
64	–	–	–	–	34.5	0.43	0.30	116.5	1.46
128	(8.00)	7.61	9.28	23.00	32.9	0.50	0.85	38.6	0.59
256	(8.00)	8.75	9.38	7.66	36.0	0.59	0.82	44.1	0.72
512	(8.00)	17.57	11.32	11.09	79.0	0.83	1.13	69.8	0.73
1000	(7.45)	6.73	10.13	4.34	71.3	1.13	0.66	108.2	1.71
1024	(1.07)	1.02	1.06	3.60	67.8	1.11	2.21	30.7	0.50
2048	(8.00)	–	58.41	13.56	–	8.12	3.75	–	2.17

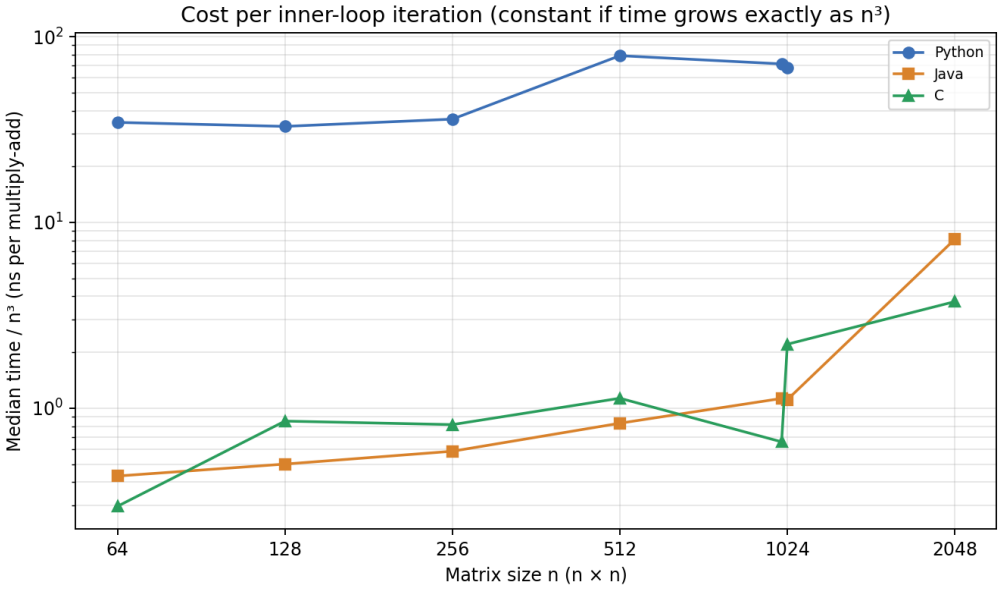


Figure 4.3: Median time divided by n^3 (nanoseconds per multiply-add). A horizontal line would mean exactly cubic growth.

A least-squares fit of $\log t = \log a + b \log n$ over $n \geq 256$ gives exponents $b = 3.42$ for Python ($n = 256$–1024), 4.11 for Java and 3.64 for C ($n = 256$–2048), all above the theoretical value of 3.

4.4. Variability

Figure 4.4 shows every measured repetition divided by the median of its configuration. For 14 of the 20 completed configurations the IQR is below 10% of the median. The largest relative IQRs are C at $n = 1000$ (38%), $n = 1024$ (31%) and $n = 256$ (16%), Java at $n = 1000$ (20%), and Python at $n = 1000$ (18%) and $n = 512$ (15%). One Java

repetition at $n = 128$ took 3.1 times its median; it is kept in the data, and the median is not affected by it.

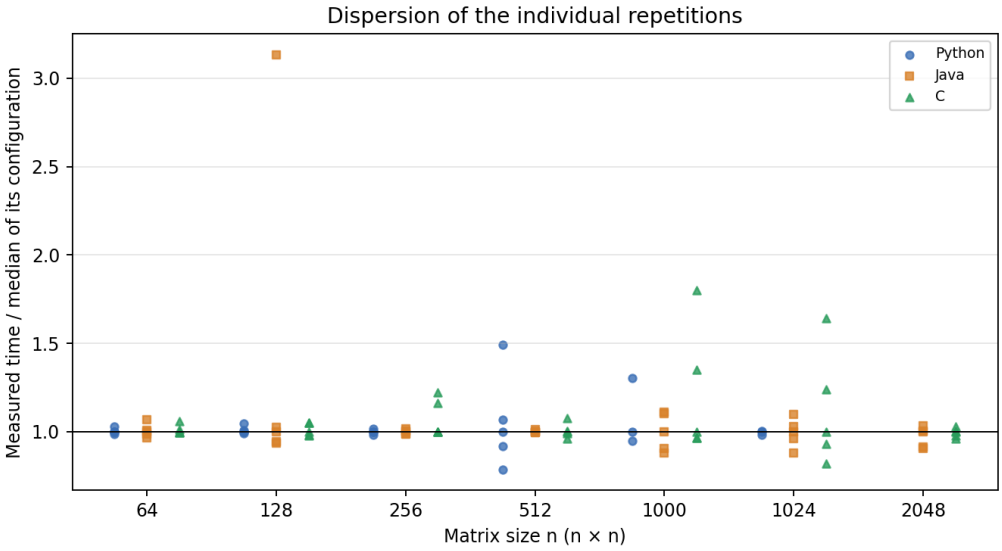


Figure 4.4: Each measured repetition divided by the median of its configuration.

4.5. Memory

Table 4.4 and Figure 4.5 report the peak working set of each process. For reference, three $n \times n$ `float64` matrices occupy 0.09 MiB for $n = 64$, 6 MiB for $n = 512$, 24 MiB for $n = 1024$ and 96 MiB for $n = 2048$.

Table 4.4: Peak working set of the process (MiB), including the language runtime. The Python value for $n = 2048$ corresponds to the run stopped by the time limit.

n	**Python**	**Java**	**C**	**Estimate $24n^2$ bytes**
64	12.1	42.1	4.3	0.1
128	14.2	43.4	4.7	0.4
256	22.5	47.6	5.6	1.5
512	51.9	66.6	9.9	6.0
1000	166.4	133.3	26.8	22.9
1024	173.1	131.8	27.9	24.0
2048	431.2 (timeout)	319.8	99.9	96.0

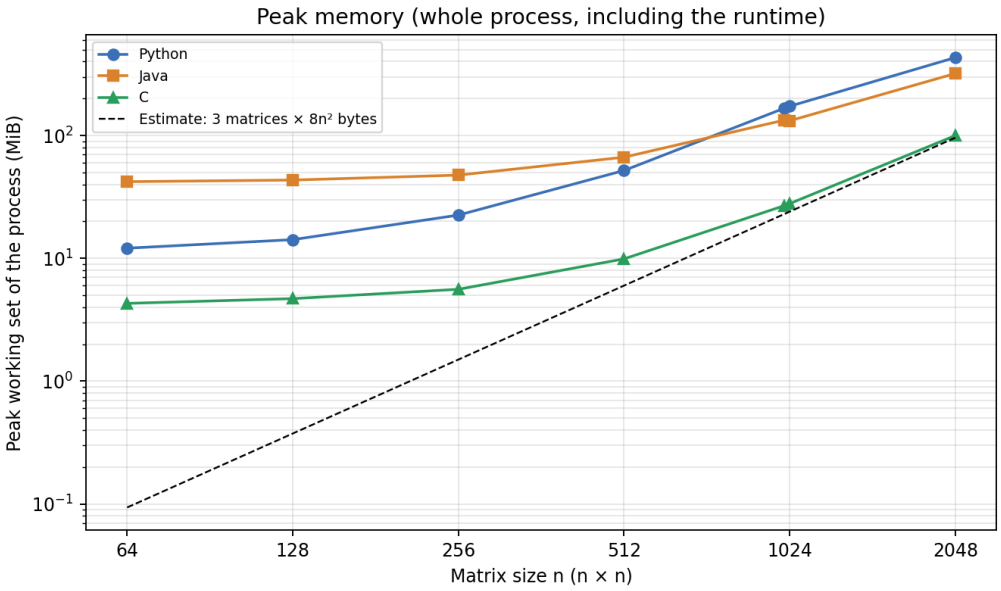


Figure 4.5: Peak working set of the process versus n, with the storage estimate of three `float64` matrices.

4.6. Limits

No configuration exceeded the memory budget of 4 GiB; the largest peak was 431 MiB. The largest feasible tested size was $n = 1024$ for Python and $n = 2048$ for Java and C. The largest attempted size was $n = 2048$ for all three languages: the first Python multiplication with $n = 2048$ did not finish within the limit of 300 s and the process was stopped, so no time is reported for it.

5. Discussion

5.1. Does the time grow as n^3? (H1)

Only partially. For small sizes the observed ratios are close to the cubic prediction of 8: Python from 64 to 256 (7.6 and 8.8), Java from 64 to 256 (9.3 and 9.4) and C from 128 to 256 (7.7). For larger sizes the time grows faster than n^3: the cost per multiply-add increases (Figure 4.3) and the fitted exponents are between 3.4 and 4.1. The algorithm still performs exactly n^3 multiply-adds, so the extra cost comes from the memory system, not from more arithmetic.

This behaviour is consistent with the size of the working data relative to the caches. The innermost loop walks down a column of B, so it touches one element of every row of B; while all of B fits in a cache, these accesses are relatively cheap. B alone occupies 2 MiB for $n = 512$ (the size of the L2 cache of one performance core), 8 MiB for $n = 1024$ (still inside the 24 MB L3 cache) and 32 MiB for $n = 2048$, which no longer fits in L3. The sharpest increases appear exactly at these points: Java, whose cost per operation stays between 0.4 and 1.1 ns up to $n = 1024$, jumps to 8.1 ns at $n = 2048$ (a ratio of 58 instead of 8), and C goes from 2.2 to 3.7 ns. In addition, for $n = 2048$ each row of B is 16 KiB long, so consecutive accesses of the inner loop always fall on different 4 KiB memory pages, which increases address-translation (TLB) misses [3]. These explanations agree with the data and with the literature [3, 4], but hardware performance counters would be needed to confirm them; they were not measured in this study.

5.2. The power-of-two effect

The most striking result is the comparison between $n = 1000$ and $n = 1024$. The cubic model predicts that 1024 should be only 7% slower than 1000. Python (+2%) and Java (+6%) behave as predicted, but C becomes 3.6 times slower (0.66 s to 2.37 s). A similar anomaly appears between $n = 64$ and $n = 128$, where C becomes 23 times slower while Python and Java grow by about 8–9.

The C matrix is one contiguous block, so the distance between B_{kj} and $B_{k+1,j}$ is exactly $8n$ bytes. When n is a power of two, this stride is a power of two, and all the elements of a column fall into a few sets of the set-associative cache, which evict each other even though the cache has plenty of free space (conflict misses) [3, 5]. In Java, each row is a separate object with a header, so the distance between rows is not exactly a power of two, and in Python the elements are pointers to objects scattered in the heap; neither layout produces this regular pattern, which explains why they are not affected. The fact that

C at $n = 1000$ is the fastest of the three languages, and that it becomes the slowest at $n = 1024$, shows that the storage layout can matter as much as the language itself.

5.3. Python versus compiled languages (H2)

Supported. Python is between 31 and 117 times slower than C and about 61 times slower than Java for $n = 1024$, in the range of one to two orders of magnitude. Its cost of 33–36 ns per multiply-add for small sizes, compared with less than 1 ns in Java and C, reflects the work of the interpreter for every iteration: reading bytecode, looking up the types of the operands, creating a new `float` object for every intermediate result and updating reference counts. This interpretation overhead dominates so strongly that Python does not show the power-of-two effect at all. Its cost per operation roughly doubles from $n = 512$ onwards, when its matrices (about 32 bytes per element including the float objects) no longer fit in the private caches. With a growth ratio of 8, a multiplication with $n = 2048$ would need roughly 580 s, consistent with the observed timeout, although this value is a prediction, not a measurement.

5.4. Java versus C (H3)

Supported for $n \leq 1024$, where Java needs between 0.50 and 1.71 times the time of C, but not for $n = 2048$, where Java is 2.2 times slower. Java was even faster than C for $n = 128$ to 512 and $n = 1024$. For $n = 128$ and 1024 this is explained by the power-of-two effect on the C layout. For $n = 256$ and 512, where C is about 1.4 times slower, the reason is not established by this study: both programs execute a scalar loop with the same operations, but the machine code generated by the HotSpot JIT compiler was not inspected, so differences in instruction scheduling, unrolling or bounds-check elimination remain possible explanations. At $n = 2048$, every access to `B[k][j]` in Java first needs the reference to row `k`, and the rows of B are separate objects; when B no longer fits in L3, these extra dependent memory accesses may explain why Java degrades more than C, but again this was not measured directly.

5.5. Memory (H4)

Supported. For large sizes the peak working set grows approximately as n^2. C is very close to the storage estimate: its baseline is about 4 MiB, and at $n = 1024$ and 2048 it uses 27.9 and 99.9 MiB against 24 and 96 MiB of data. The JVM needs about 42 MiB even for the smallest matrices, and at $n = 2048$ it reaches 320 MiB, more than three times the data. Part of this is the heap of a garbage-collected runtime: every repetition

allocates a new result matrix, and the previous ones are released only when the collector runs; the input files are also read into temporary byte arrays. Python uses about 6–7 times the size of the data at $n = 1024$, because each element is an 8-byte pointer to a separate 24-byte `float` object instead of an 8-byte value. The metric includes the whole process, so the comparison mixes data and runtime overhead; it should not be read as the size of the matrices alone.

5.6. Variability and the hybrid processor

The preliminary run without affinity shows that, on a hybrid processor, the operating system can move a single-threaded benchmark between core types with very different speeds. This made some medians more than twice as high and some repetitions several times slower than others, which would have led to wrong conclusions (for example, Java $n = 2048$ exceeded the time limit in that run). Pinning every process to a performance core removed most of this variability: 14 of 20 configurations have an IQR below 10%. The remaining variability is concentrated in the sizes where the caches are under the most pressure ($n = 512$ to 1024), which suggests interference from other processes that share the L3 cache and memory bandwidth, and changes of the clock frequency under the *Balanced* power plan.

5.7. Limitations

- A single laptop was used, with the *Balanced* power plan and normal background activity (including cloud synchronisation). Clock frequency and other processes could not be fully controlled.
- Each process was pinned to one logical processor, but the other hardware thread of the same core was not reserved, and no hardware performance counters (cache misses, TLB misses) were collected. The explanations based on the memory hierarchy are therefore supported by the timing patterns, not by direct measurements.
- Costly configurations used only three repetitions, so their IQR is less reliable.
- The memory metric is the peak working set of the whole process; it does not separate matrix storage from runtime overhead, and for Java it depends on when the garbage collector runs.
- The JIT-compiled code of Java was not inspected, so the reason why Java is faster than C for some sizes is not fully resolved.

6. Conclusions

Within the tested implementations, settings and machine, this study establishes that:

- The same triple loop, with identical inputs and bit-identical results, needs 72.8 s in Python, 1.19 s in Java and 2.37 s in C for $n = 1024$. Python is one to two orders of magnitude slower than the compiled languages and could not complete $n = 2048$ within 300 s.
- The time follows the cubic model only while the data fits in the caches. Beyond that point the cost per operation grows, and the fitted exponents over $n = 256$–2048 are 3.4–4.1.
- The storage layout interacts strongly with the matrix size: the contiguous C layout is 3.6 times slower at $n = 1024$ than at $n = 1000$, an effect that the Java and Python layouts do not show. Consequently, no language is faster than another for every size: C was the fastest at $n = 64$, 1000 and 2048, and Java at $n = 128$–512 and 1024.
- Memory use grows as n^2; C stays close to the size of the data, while the JVM adds a fixed overhead and heap growth, and Python needs several times the size of the data because of boxed numbers.
- On a hybrid processor, controlling the CPU affinity is essential: without it, the results were up to 2.7 times slower and much more variable.

What remains unresolved is the exact cause of the differences between Java and C at $n = 256$, 512 and 2048, which would require inspecting the generated machine code and collecting hardware counters. These results also motivate Assignment 2: the cache effects observed here suggest that loop reordering and blocking should reduce the cost per operation of the basic algorithm.

AI Usage Statement

In accordance with the course policy, a generative AI assistant was used to support the planning of the experiment, the development and debugging of the benchmark scripts, and the language revision of this report. All the measurements were executed on the author's computer, and the author checked the code, the results and the references and is responsible for the conclusions.

Reproducibility Materials

All materials are in the folder `assignment1` of the repository https://github.com/Claudia1771/bigdata-matrix-multiplication:

- `src/`: the Python, Java and C implementations.
- `config.json`: seed, sizes, repetitions, warm-ups, budgets, compiler and JVM flags, CPU affinity and tolerances.
- `scripts/`: `generate_inputs.py`, `verify.py`, `core_probe.py`, `run_experiment.py` and `analyze.py`.
- `data/manifest.csv`: seeds and SHA-256 hashes of the input files (the binary inputs are regenerated by `generate_inputs.py`).
- `results/`: raw measurements, memory, correctness tests, environment, summaries, figures and the preliminary run without affinity.
- `README.md`: exact build and run commands and dependency versions.

References

[1] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins Univ. Press, 2013.

[2] C. E. Leiserson, N. C. Thompson, J. S. Emer, B. C. Kuszmaul, B. W. Lampson, D. Sanchez, and T. B. Schardl, "There's plenty of room at the Top: What will drive computer performance after Moore's law?," *Science*, vol. 368, no. 6495, Art. no. eaam9744, 2020.

[3] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., Tech. Rep., Nov. 2007.

[4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[5] M. S. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*, 1991, pp. 63–74.

[6] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Softw.*, vol. 34, no. 3, Art. no. 12, 2008.

[7] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, 2007, pp. 57–76.

[8] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results," in *Proc. Int. Conf. High Performance Computing, Networking, Storage and Analysis (SC15)*, 2015, Art. no. 73.

[9] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[10] J. D. Darcy, "JEP 306: Restore always-strict floating-point semantics," OpenJDK, 2021. [Online]. Available: https://openjdk.org/jeps/306